

GIT & GITHUB BOOTCAMP

Lesson 2

Git Basics

Now that Git is installed and configured, it's time to actually use it. This lesson covers the core concepts and the everyday commands you'll use in every Git project for the rest of your career.

What You'll Learn

By the end of this lesson, students will be able to:

1. Explain what a repository is.
2. Describe Git's three areas: the working directory, the staging area, and the repository.
1. Understand what a commit is and why commit history matters.
2. Use the core commands: git init, git status, git diff, git add, git commit, and git log.

What is a Repository?

Every Git project lives inside a repository (often shortened to "repo").

A repository is just a normal project folder that Git is watching. Inside it, Git creates a hidden folder called `.git` where it quietly stores the entire history of your project.

Definition

Repository (Repo):

A folder that Git tracks, containing your project files plus a hidden `.git` folder that stores the project's complete history.

You don't need to ever touch the `.git` folder yourself — Git manages it. But know that if you delete `.git`, you delete the history.

What's Inside a Repository

```
my-project/      ← the repository
├── .git/         ← hidden: Git's history and database (don't touch)
├── index.html    ← your files
├── style.css
└── README.md
```

Everything except `.git` is your actual project. The `.git` folder is the "time machine" working behind the scenes.

The Three Areas of Git

This is the most important concept in this lesson. Git organizes your work into three areas. Understanding them makes every command make sense.

Area	What It Is
Working Directory	The actual files you see and edit on your computer.
Staging Area	A "waiting room" where you gather changes you're ready to save.
Repository (.git)	The permanent history where saved snapshots (commits) live.

How Changes Flow Through the Areas

A change moves through these areas in a clear path:

```
Working Directory (you edit files) → Staging Area (changes you pick) → Repository (saved forever)
                                git add                                git commit
```

1. You edit files in the working directory.
2. You run `git add` to move chosen changes into the staging area.
3. You run `git commit` to permanently save them into the repository.

Why a Staging Area?

Beginners often ask: why not just save everything at once?

The staging area lets you choose exactly what goes into each snapshot. Maybe you fixed two unrelated things — a typo and a bug. You can stage and commit them separately, so your history stays clean and meaningful.

Think of staging like a shopping cart: you add the items you want, review the cart, and only then "check out" (commit). You don't have to buy everything in the store at once.

What is a Commit?

A commit is the heart of Git. It is a single saved snapshot of your project at a moment in time.

Definition

Commit:

A permanent snapshot of your staged changes, saved in the repository with a unique ID, an author, a timestamp, and a message describing what changed.

Each commit answers three questions:

- What changed?

- Who changed it?
- Why did they change it? (the commit message)

Anatomy of a Commit

```
commit a1b2c3d4...      ← unique ID (hash)
Author: Your Name <you@example.com>
Date:   Wed Jun 18 2026

    Add contact form to homepage  ← commit message
```

A good commit message is short, clear, and written in the present tense, describing what the change does.

Good Message	Poor Message
Add login button to navbar	stuff
Fix crash when list is empty	asdf
Update README with setup steps	changes

Common mistake

Writing vague messages like "update" or "fix". Six months later, nobody (including you) will know what that commit did.

Commit History

Because every commit is saved, your project builds up a history — a timeline of snapshots from the very first commit to the latest.

```
(oldest) → commit 1 → commit 2 → commit 3 → (newest)
           "Initial    "Add about  "Fix footer
           commit"     page"       spacing"
```

This history is your project's time machine. You can:

- See exactly how the project evolved.
- Find when (and why) a bug was introduced.
- Return to any earlier version if needed.

The Core Commands

Now let's connect the concepts to real commands. These six commands are the foundation of daily Git use.

git init — Start a Repository

Run this once inside a project folder to turn it into a Git repository. It creates the hidden .git folder.

```
git init

Initialized empty Git repository in /home/you/my-project/.git/
```

git status — Check What's Going On

git status is the command you'll run most often. It tells you the current state of your three areas: what's changed, what's staged, and what's not yet tracked.

```
git status

On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html

nothing added to commit but untracked files present
```

Tip

When in doubt, run git status. It almost always tells you what to do next.

git diff — See Exactly What Changed

git status tells you which files changed. git diff shows you what changed inside those files — line by line.

```
git diff
```

This compares your working directory to the staging area. It shows unstaged changes — edits you haven't run git add on yet.

Example output after editing index.html:

```
diff --git a/index.html b/index.html
index e69de29..3b18e51 100644
--- a/index.html
+++ b/index.html
@@ -1,3 @@
+<h1>Hello World</h1>
+<p>Welcome to my project.</p>
```

Lines starting with + were added. Lines starting with - were removed.

To review changes you've already staged (staging area vs. the last commit), use:

```
git diff --staged
```

Tip

Run `git diff` before you stage, and `git diff --staged` before you commit. That way you always know exactly what you're about to save.

git add — Stage Changes

`git add` moves changes from the working directory into the staging area.

```
# Stage a single file
git add index.html

# Stage multiple specific files
git add index.html style.css

# Stage everything that changed
git add .
```

After running `git add`, run `git status` again — the file will now appear under "Changes to be committed."

git commit — Save a Snapshot

`git commit` permanently saves everything in the staging area as a new commit. The `-m` flag lets you write the message inline.

```
git commit -m "Add homepage layout"

[main (root-commit) a1b2c3d] Add homepage layout
1 file changed, 20 insertions(+)
```

Watch out

Only staged changes are committed. If you edited a file but forgot to `git add` it, that change will not be in the commit.

git log — View the History

`git log` shows the list of commits, newest first.

```
git log

commit a1b2c3d4e5 (HEAD -> main)
Author: Your Name <you@example.com>
Date:   Wed Jun 18 16:00:00 2026
```

For a shorter, cleaner view, use:

```
git log --oneline  
  
a1b2c3d Add homepage layout
```

Putting It All Together

Here is a complete, realistic first workflow — from empty folder to first commit:

```
# 1. Create and enter a project folder  
mkdir my-project  
cd my-project  
  
# 2. Turn it into a Git repository  
git init  
  
# 3. Create a file (or add your project files)  
echo "Hello World" > index.html  
  
# 4. Check the status  
git status  
  
# 5. See what changed (after you edit a tracked file)  
git diff  
  
# 6. Stage the file  
git add index.html  
  
# 7. Review what you're about to commit  
git diff --staged  
  
# 8. Commit the snapshot  
git commit -m "Add initial homepage"  
  
# 9. View your history  
git log --oneline
```

Congratulations — that's the core loop of Git. You'll repeat edit → diff → add → commit thousands of times.

The Everyday Cycle

```
edit files → git status → git add → git commit → (repeat)  
      ↑_____|  
      |  
      |
```

Summary

- A repository is a project folder that Git tracks, using a hidden `.git` folder.
- Git has three areas: the working directory (your files), the staging area (changes you've selected), and the repository (saved history).
- A commit is a permanent snapshot with an ID, author, timestamp, and a message.
- The commit history is your project's timeline and time machine.
- Core commands: `git init` (start), `git status` (check), `git diff` (review changes), `git add` (stage), `git commit` (save), `git log` (history).
- The everyday loop is `edit` → `diff` → `add` → `commit`.

End of Lesson 2